

Transaksi Database dan Relasi Tingkat Lanjut: GORM Relational Query & Transaction Control di Go

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Untuk membangun REST API finansial atau e-commerce berskala enterprise, relasi data antar-tabel dan integritas data mutlak diperlukan. Anda harus memastikan bahwa operasi multi-tabel (seperti pengurangan stok barang sekaligus pembuatan faktur tagihan) harus bersifat **Atomik** (seluruhnya berhasil, atau tidak sama sekali). Kegagalan di tengah jalan tanpa *rollback* akan menyebabkan inkonsistensi data yang sangat fatal bagi bisnis.

Modul ini akan membahas pemodelan relasi tingkat lanjut di GORM, pemrosesan kueri relasional menggunakan *Preloading*, serta implementasi manajemen **Transaksi Database** (baik secara otomatis maupun manual) lengkap dengan mekanisme *Row-Level Locking* untuk mencegah masalah balapan data (*race condition*).

Sesi 1: Relasi & Asosiasi GORM (One-to-Many & Many-to-Many) (45 Menit)

GORM mempermudah pemetaan hubungan relasional antar-tabel database menggunakan struct Go.

1. Relasi One-to-Many (Satu ke Banyak)

Relasi *One-to-Many* terjadi saat satu model memiliki banyak model anak.

- **Contoh:** Satu `User` dapat memiliki banyak `Order` .
- **Definisi Model:**

```
type User struct {
    ID      uint      `gorm:"primaryKey" json:"id"`
    Email   string    `gorm:"type:varchar(100);uniqueIndex;not null"
    json:"email"`
    Orders  []Order   `gorm:"foreignKey:UserID" json:"orders,omitempty" // Has
    Many
}

type Order struct {
    ID      uint      `gorm:"primaryKey" json:"id"`
```

```

        UserID      uint      `json:"user_id"` // Foreign Key
        OrderNumber string    `gorm:"type:varchar(50);uniqueIndex;not null"
        json:"order_number"`
        TotalAmount float64    `gorm:"type:numeric(12,2)" json:"total_amount"`
    }

```

2. Relasi Many-to-Many (Banyak ke Banyak)

Relasi *Many-to-Many* menghubungkan dua model melalui sebuah tabel perantara (*junction table*).

- **Contoh:** Hubungan antara `Order` dengan `Product` melalui tabel detail item pesanan (`order_items`).
- **Definisi Model GORM:**

```

type Product struct {
    ID      uint      `gorm:"primaryKey" json:"id"`
    SKU     string    `gorm:"type:varchar(50);uniqueIndex;not null" json:"sku"`
    Name    string    `gorm:"type:varchar(100);not null" json:"name"`
    Price   float64   `gorm:"type:numeric(12,2);not null" json:"price"`
    Stock   int       `gorm:"type:int;not null" json:"stock"`
}

type OrderItem struct {
    OrderID   uint      `gorm:"primaryKey" json:"order_id"`
    ProductID uint      `gorm:"primaryKey" json:"product_id"`
    Quantity  int       `gorm:"type:int;not null" json:"quantity"`
    Price     float64   `gorm:"type:numeric(12,2);not null" json:"price"`
    Product   Product   `gorm:"foreignKey:ProductID" json:"product"` // Belongs To
}

```

Catatan: Kita mendefinisikan `OrderItem` secara eksplisit sebagai model tersendiri agar dapat menyimpan kolom kustom tambahan seperti `quantity` dan `price` pada tabel perantara.

3. Mengambil Data Relasional via Preloading

GORM melakukan pencarian data secara *Lazy Loading* (tidak mengambil data relasi secara default demi performa). Untuk mengambil data asosiasi, kita wajib menggunakan fungsi `Preload` (*Eager Loading*).

- **Mengambil Order beserta Data User-nya:**

```

var order Order
db.Preload("User").First(&order, 1)

```

- **Preloading Bertingkat (Nested Preloading):** Kita ingin mengambil data `Order`, data `User` pemesan, daftar `OrderItem`-nya, sekaligus detail `Product` di dalam masing-masing item

tersebut.

```
var order Order
db.Preload("User").Preload("Items.Product").First(&order, 1)
```

Sesi 2: Konsep & Implementasi Transaksi GORM (45 Menit)

Transaksi database adalah kumpulan satu atau lebih perintah SQL yang dieksekusi sebagai satu unit logis. Transaksi wajib memenuhi kaidah **ACID** (Atomicity, Consistency, Isolation, Durability).

- **Atomicity**: Semua operasi sukses, atau semua operasi dibatalkan (*Rollback*).

1. Transaksi Otomatis (`db.Transaction`)

GORM menyediakan method `db.Transaction` yang sangat aman. Jika fungsi di dalamnya mengembalikan error (`return err`), GORM otomatis melakukan `ROLLBACK`. Jika mengembalikan `nil`, GORM otomatis melakukan `COMMIT`.

```
err := db.Transaction(func(tx *gorm.DB) error {
    // A. Simpan data pesanan baru
    newOrder := Order{UserID: 1, OrderNumber: "INV-999", TotalAmount: 500000}
    if err := tx.Create(&newOrder).Error; err != nil {
        return err // GORM akan otomatis Rollback transaksi
    }

    // B. Simpan item pesanan
    item := OrderItem{OrderID: newOrder.ID, ProductID: 2, Quantity: 1, Price:
500000}
    if err := tx.Create(&item).Error; err != nil {
        return err // GORM akan otomatis Rollback transaksi
    }

    return nil // Sukses! GORM akan otomatis Commit transaksi
})
```

2. Transaksi Manual (`db.Begin`)

Jika Anda memerlukan kontrol penuh di dalam alur handler Anda (misal mengalirkan langkah-langkah di file terpisah), Anda dapat menggunakan transaksi manual:

```
// 1. Memulai transaksi
tx := db.Begin()

// 2. Lakukan operasi (Wajib menggunakan objek 'tx', bukan 'db!')
```

```

if err := tx.Create(&order).Error; err != nil {
    tx.Rollback() // Batalan transaksi
    return
}

if err := tx.Create(&item).Error; err != nil {
    tx.Rollback() // Batalan transaksi
    return
}

// 3. Commit jika semua langkah sukses
tx.Commit()

```

WARNING

Transaction Object Leak: Kesalahan paling fatal developer Go saat menggunakan transaksi adalah tetap memanggil objek koneksi global `db.Create()` atau `db.Update()` di dalam blok transaksi. Pastikan Anda selalu memanggil variabel transaksi lokal (`tx`) untuk setiap kueri di dalam blok transaksi.

Sesi 3: ACID & Concurrency Control (Row Locking) (45 Menit)

Dalam sistem e-commerce dengan lalu lintas request tinggi, masalah *race condition* sering terjadi saat beberapa user membeli produk yang sama secara bersamaan.

- **Masalah:** Dua transaksi membaca stok produk yang sama bernilai 1. Kedua transaksi melihat stok mencukupi, lalu keduanya sama-sama mengurangi stok menjadi 0 dan membuat pesanan. Akibatnya, terjadi *overselling* (barang terjual 2x padahal stok hanya 1).
- **Solusi:** Menggunakan mekanisme kunci baris data (*Row-Level Locking*) menggunakan klausa `FOR UPDATE`.

Di GORM, kita dapat menerapkan penguncian baris data dengan klausa `clause.Locking`:

```

import "gorm.io/gorm/clause"

var product Product
// Mengunci baris produk ini agar transaksi lain tidak bisa membaca/mengubahnya hingga
transaksi ini selesai
err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).First(&product, id).Error

```

Sesi 4: Praktikum Mandiri: REST API Pemesanan E-Commerce dengan Transaksi & Relasi Kompleks (45 Menit)

Sekarang kita akan membangun REST API pemesanan e-commerce terintegrasi. Praktikum ini menggunakan Gin, GORM, dan database PostgreSQL (ecommercedb port 5434). Kita akan menerapkan transaksi aman untuk checkout barang, memotong stok, mengunci baris data produk, dan mencetak invoice dengan kueri preloading bertingkat.

Implementasi Kode (main.go)

```
package main

import (
    "errors"
    "fmt"
    "net/http"
    "strconv"
    "time"

    "github.com/gin-gonic/gin"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
    "gorm.io/gorm/clause"
)

// =====
// 1. DEFINISI MODEL & DATABASE SCHEMAS
// =====

type User struct {
    ID          uint       `gorm:"primaryKey" json:"id"`
    Email       string     `gorm:"type:varchar(100);uniqueIndex;not null" json:"email"`
    CreatedAt   time.Time `json:"created_at"`
}

type Product struct {
    ID          uint       `gorm:"primaryKey" json:"id"`
    SKU         string     `gorm:"type:varchar(50);uniqueIndex;not null" json:"sku"`
    Name        string     `gorm:"type:varchar(100);not null" json:"name"`
    Price       float64    `gorm:"type:numeric(12,2);not null" json:"price"`
    Stock       int        `gorm:"type:int;not null;default:0" json:"stock"`
    CreatedAt   time.Time `json:"created_at"`
}

type Order struct {
    ID          uint       `gorm:"primaryKey" json:"id"`
    UserID      uint       `json:"user_id"`
}
```

```

        User        User        `gorm:"foreignKey:UserID" json:"user,omitEmpty"`
        OrderNumber string      `gorm:"type:varchar(50);uniqueIndex;not null"
json:"order_number"`
        TotalAmount float64    `gorm:"type:numeric(12,2)" json:"total_amount"`
        Status      string      `gorm:"type:varchar(20);default:'COMPLETED'"
json:"status"`
        Items       []OrderItem `gorm:"foreignKey:OrderID" json:"items"`
        CreatedAt    time.Time  `json:"created_at"`
    }

    type OrderItem struct {
        OrderID    uint    `gorm:"primaryKey" json:"order_id"`
        ProductID  uint    `gorm:"primaryKey" json:"product_id"`
        Product    Product `gorm:"foreignKey:ProductID" json:"product,omitEmpty"`
        Quantity   int     `gorm:"type:int;not null" json:"quantity"`
        Price      float64 `gorm:"type:numeric(12,2);not null" json:"price"`
    }

    var db *gorm.DB

    func initDatabase() {
        dsn := "host=localhost user=postgres password=secret45 dbname=ecommercedb
port=5434 sslmode=disable TimeZone=Asia/Jakarta"
        var err error
        db, err = gorm.Open(postgres.Open(dsn), &gorm.Config{})
        if err != nil {
            panic("Gagal terkoneksi ke database: " + err.Error())
        }

        // Jalankan Auto Migration untuk semua model relasional
        _ = db.AutoMigrate(&User{}, &Product{}, &Order{}, &OrderItem{})

        // Isi data dummy awal jika database kosong
        var count int64
        db.Model(&User{}).Count(&count)
        if count == 0 {
            userDummy := User{Email: "budi.santoso@gmail.com"}
            db.Create(&userDummy)

            prod1 := Product{SKU: "SKU-EL-01", Name: "Keyboard Mechanical", Price:
800000.00, Stock: 10}
            prod2 := Product{SKU: "SKU-EL-02", Name: "Mouse Wireless Gaming",
Price: 450000.00, Stock: 15}
            db.Create(&prod1)
            db.Create(&prod2)
        }
    }

    // =====
    // 2. REQUEST VALIDATION STRUCTS
    // =====

    type OrderItemInput struct {

```

```

        ProductID uint `json:"product_id" binding:"required"`
        Quantity  int  `json:"quantity" binding:"required,gt=0"`
    }

    type CheckoutInput struct {
        UserID uint          `json:"user_id" binding:"required"`
        Items  []OrderItemInput `json:"items" binding:"required,dive"`
    }

    // =====
    // 3. MAIN CONTROLLERS & ROUTES
    // =====

    func main() {
        initDatabase()

        r := gin.Default()

        api := r.Group("/api/v1")
        {
            // POST /api/v1/orders - Transaksi Pemesanan (Checkout)
            api.POST("/orders", func(c *gin.Context) {
                var input CheckoutInput
                if err := c.ShouldBindJSON(&input); err != nil {
                    c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})

                    return
                }

                var newOrder Order

                // Memulai Transaksi Database secara otomatis
                errTx := db.Transaction(func(tx *gorm.DB) error {
                    // A. Validasi keberadaan User
                    var user User
                    if err := tx.First(&user, input.UserID).Error; err !=
nil {
                        return fmt.Errorf("user dengan ID %d tidak
ditemukan", input.UserID)
                    }

                    // B. Inisialisasi total tagihan dan items array
                    var totalAmount float64
                    var orderItems []OrderItem

                    // C. Generate nomor invoice unik
                    orderNumber := fmt.Sprintf("INV/%s/%d",
time.Now().Format("20060102"), time.Now().UnixNano()%10000)

                    // D. Simpan record order utama terlebih dahulu agar
mendapatkan OrderID

                    newOrder = Order{
                        UserID:      user.ID,

```

```

        OrderNumber: orderNumber,
        TotalAmount: 0, // di-update setelah loop

kalkulasi item
        Status:      "COMPLETED",
    }
    if err := tx.Create(&newOrder).Error; err != nil {
        return err
    }

    // E. Loop memproses setiap item produk
    for _, item := range input.Items {
        var product Product

        // Gunakan Row-Level Locking (FOR UPDATE)
        untuk menghindari race condition stok
        errLock := tx.Clauses(clause.Locking{Strength:
"UPDATE"}).First(&product, item.ProductID).Error
        if errLock != nil {
            return fmt.Errorf("produk ID %d tidak
ditemukan", item.ProductID)
        }

        // Validasi kecukupan stok barang
        if product.Stock < item.Quantity {
            return fmt.Errorf("stok barang '%s'
tidak mencukupi (Sisa: %d, Permintaan: %d)",
                                product.Name, product.Stock,
            item.Quantity)
        }

        // Update stok produk di database
        newStock := product.Stock - item.Quantity
        if err := tx.Model(&product).Update("stock",
newStock).Error; err != nil {
            return err
        }

        // Hitung subtotal dan susun objek item
        pesanan
        subtotal := product.Price *
float64(item.Quantity)
        totalAmount += subtotal

        orderItem := OrderItem{
            OrderID:  newOrder.ID,
            ProductID: product.ID,
            Quantity: item.Quantity,
            Price:    product.Price,
        }

        // Simpan detail item ke database
        if err := tx.Create(&orderItem).Error; err !=
nil {

```

```

        return err
    }

    orderItems = append(orderItems, orderItem)
}

// F. Update total_amount akhir pada tabel orders
newOrder.TotalAmount = totalAmount
newOrder.Items = orderItems
if err := tx.Model(&newOrder).Update("total_amount",
totalAmount).Error; err != nil {
    return err
}

return nil // Transaksi sukses! GORM melakukan COMMIT
secara otomatis
})

// Cek apakah transaksi gagal dan ter-rollback
if errTx != nil {
    c.JSON(http.StatusBadRequest, gin.H{"error":
errTx.Error()})

    return
}

// Mengembalikan response sukses beserta data invoice pesanan
c.JSON(http.StatusOK, gin.H{
    "message": "Transaksi checkout berhasil diproses!",
    "data":    newOrder,
})
})

// GET /api/v1/orders/:id - Mendapatkan Faktur/Invoice Lengkap dengan
nested Preloading
api.GET("/orders/:id", func(c *gin.Context) {
    idStr := c.Param("id")
    id, _ := strconv.Atoi(idStr)

    var order Order
    // Melakukan eager loading bertingkat: Memuat User dan Items
    beserta Product di dalamnya
    err :=
db.Preload("User").Preload("Items.Product").First(&order, id).Error
    if err != nil {
        if errors.Is(err, gorm.ErrRecordNotFound) {
            c.JSON(http.StatusNotFound, gin.H{"error":
"Faktur pesanan tidak ditemukan"})

            return
        }
        c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})

        return
    }
}

```

```
        c.JSON(http.StatusOK, gin.H{"data": order})
    })
}

r.Run(":8080")
}
```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemrograman transaksi dan relasi di bawah ini untuk memperkuat pemahaman Anda mengenai integrasi sistem keamanan database relasional di Go.

Latihan 1: Transaksi Pembatalan Pesanan dengan Restock Otomatis

Deskripsi Soal: Ketika pembeli membatalkan pesanan, sistem harus mengembalikan stok barang yang dibeli ke kondisi semula (*restock*) secara otomatis dan memperbarui status pesanan menjadi 'CANCELLED'. Proses ini wajib dibungkus dalam transaksi database tunggal agar tidak terjadi inkonsistensi data.

Kriteria Implementasi:

1. Buatlah endpoint `PUT /api/v1/orders/:id/cancel` pada program Gin Anda.
2. Di dalam handlernya, jalankan transaksi database kustom:
 - Ambil data `Order` berdasarkan ID beserta seluruh list `OrderItem` pendukungnya (gunakan `Preload("Items")`).
 - Pastikan status pesanan saat ini belum 'CANCELLED'. Jika sudah, gagalkan transaksi dengan mengembalikan pesan error.
 - Lakukan perulangan (*loop*) untuk setiap item di pesanan tersebut. Kembalikan jumlah kuantitas yang dibeli ke kolom `stock` masing-masing produk terkait di tabel `products`.
 - Ubah kolom `status` pada tabel `orders` menjadi 'CANCELLED'.
3. Kirim respons JSON sukses yang menyertakan data status order terbaru yang sudah ter-update.

Latihan 2: Nested Preloading & Custom Join Queries

Deskripsi Soal: Buat kueri analitik sederhana menggunakan GORM untuk mendapatkan rangkuman total kuantitas produk yang terjual untuk kategori elektronik.

Kriteria Implementasi:

1. Buatlah endpoint `GET /api/v1/analytics/popular-products` pada aplikasi Anda.
2. Gunakan GORM Query Builder untuk menggabungkan tabel `order_items` dengan `products`.
3. Lakukan agregasi jumlah penjualan produk menggunakan fungsi `Select` dan `Group` bawaan GORM:

```
// Contoh visual kueri GORM
db.Table("order_items").
  Select("product_id, SUM(quantity) as total_sold").
  Group("product_id")
```

4. Kirim response list data produk terpopuler beserta akumulasi jumlah penjualannya ke klien.